Mar 5, 2025

Class #8:

📌 Limitations of RNNs and LSTMs

- Sequential Processing (Slow Training) - Since RNNs and LSTMs pass hidden states sequentially, they cannot be **parallelised**, leading to longer training times.
- Limited Context Retention - RNNs and LSTMs cannot efficiently retain **long-term context**, whereas Transformers use **self-attention** to maintain relationships between distant words.
- Fixed-Length Memory Bottleneck - If the sequence is long, earlier information **gets forgotten**. Transformers allow direct attention to all tokens at once, **preserving full context**.

Join @hmaracollege for more free courses🙃

📌 Transformer Architecture

The Transformer is a deep learning model architecture introduced in the paper **"Attention is All You Need"**. It has revolutionized Natural Language Processing (NLP), computer vision, and AI applications by replacing traditional Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs).

Unlike RNNs and LSTMs, which process data sequentially, Transformers process input in **parallel**, making them **faster** and **more efficient** for large datasets.

✒️ Key Features/Benefits of Transformer Architecture

- Parallel Processing - Unlike RNNs and LSTMs, which process data sequentially, Transformers process the entire **input sequence at once**. This makes them significantly **faster** and **more scalable**.
- Self-Attention Mechanism - Helps capture long-range dependencies in text. Can focus on **relevant words**, regardless of their **position**.

✒️ Why Matrices Don't Naturally Capture Sequence/Order?

- In matrix operations **swapping two rows (tokens) does not change how the operation works**.
- No **sequential dependencies** are built into the attention mechanism itself.

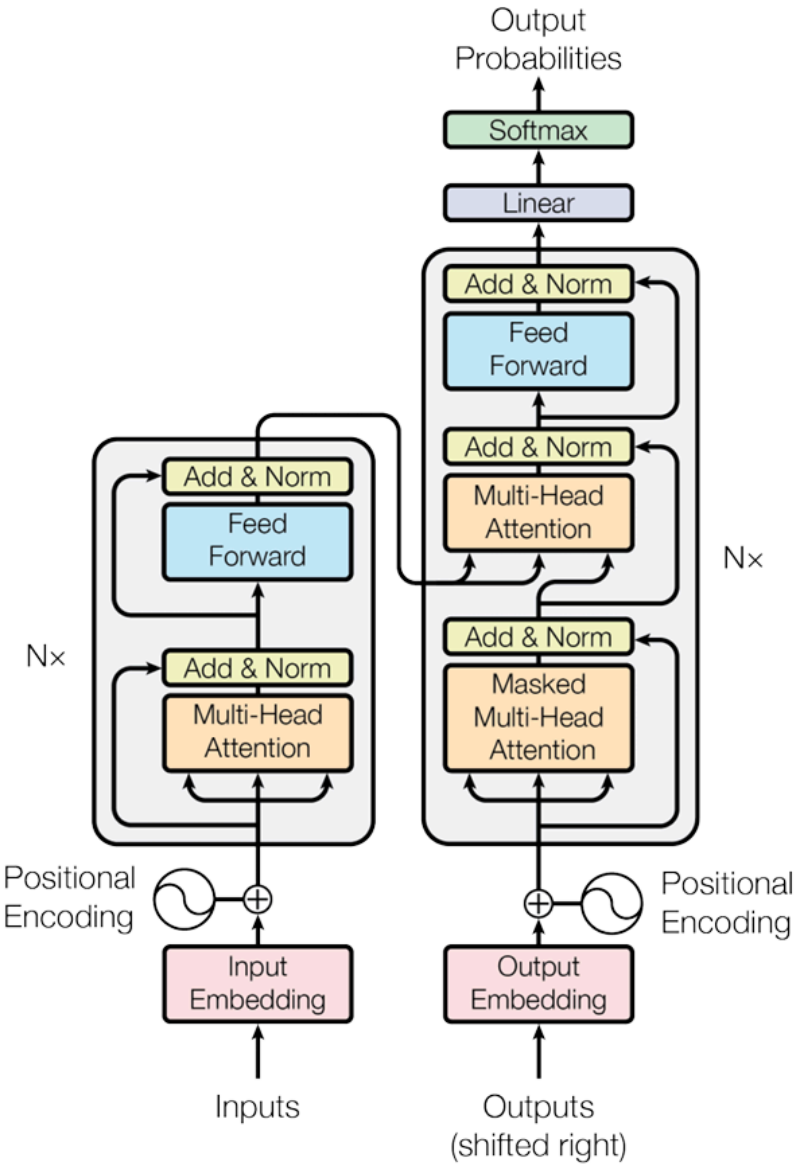


Figure 1: The Transformer - model architecture.

Join @hmaracollege for more free courses🙃

✒️ Self Attention Matrix - The **Self-Attention Matrix** is the core component of the Transformer model that helps it understand relationships between words (tokens) in a sentence, **regardless of their position**. Unlike the processing of sequences **step-by-step**, Self-Attention allows Transformers to process **all tokens at once** while determining their contextual importance.

A **Self-Attention Matrix** is an $n \times n$ **matrix**, where n **is the number of tokens** in a sentence, and each value represents how much attention one token pays to another.

✒️ Why is Self-Attention Important?

- Captures Long-Range Dependencies - **"The cat sat on the mat, and it was happy."** The word **"it" refers to "cat"**. Self-Attention helps the model understand this connection, even if **"it" is far from "cat"**.
- Parallel Computation - Since attention can be computed for all words **simultaneously**, Transformers train much faster than RNNs/LSTMs.

📌 Positional Encoding

✒️ Goal of Positional Encoding- The main goal is to find a vector of unique encoding for different positions, of the same dimension as the dimensions of the word.

Join @hmaracollege for more free courses🙃

✒️ Positional Encoding Intuition - **Positional Encoding (PE)** in Transformers is designed to **provide each word in a sequence with a unique positional representation** while also **capturing relative distances between tokens effectively**.

Each **embedding dimension oscillates at a different frequency:**

- Lower dimensions (smaller i) $\rightarrow$ Higher frequencies - Captures fine-grained position changes for nearby words.
- Higher dimensions (larger i) $\rightarrow$ Lower frequencies - Captures long-range positional relationships.

This multi-scale frequency variation allows the Transformer to compare words both locally and globally.

Since each position pos generates different values for sin and cos, no two positions have the same encoding, making each position distinct.

Example:

If $d = 4$, and we compute PE for **pos = 1, 2, 3**, we get:

Position	PE(0) (High Freq)	PE(1) (Medium Freq)	PE(2) (Low Freq)	PE(3) (Very Low Freq)
1	sin(1) = 0.84	sin(0.1) = 0.10	sin(0.001) ≈ 0.001	sin(0.00001) ≈ 0.00001
2	sin(2) = 0.91	sin(0.2) = 0.20	sin(0.002) ≈ 0.002	sin(0.00002) ≈ 0.00002
3	sin(3) = 0.14	sin(0.3) = 0.30	sin(0.003) ≈ 0.003	sin(0.00003) ≈ 0.00003

🔹 **Observation:**

- The **high-frequency columns change rapidly** (fine-grained local positioning).
- The **low-frequency columns change slowly** (global positioning across the sentence).
- **No two positions have the same encoding**, ensuring uniqueness.

✒️ Positional Encoding Formula - Since Transformers don't have a built-in sense of order (like RNNs do), **we encode positional information directly into the word embeddings using sinusoidal functions.**

Join @hmaracollege for more free courses🙃

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

- pos = Position of the token in the sequence (0, 1, 2, ...).
- i = The dimension index (half for sine, half for cosine).
- d = Total embedding dimension (e.g., 512 in original Transformer).
- 10000 = A scaling factor to ensure smooth variations.

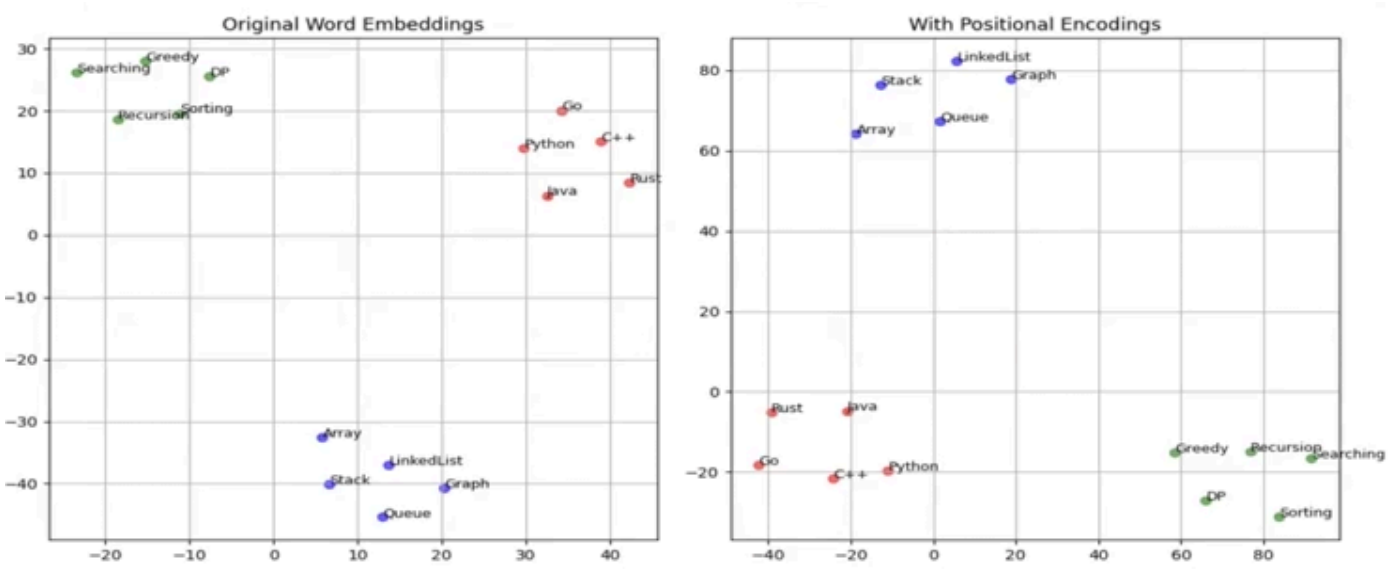
✒️ Why Sinusoidal Functions?

Relative positioning was introduced so that nearby words have similar encodings. They **generalize to unseen sequence lengths** better than fixed position indices.

Instead of using simple numbers like 1,2,3,... for positions, we use sinusoidal functions because it creates a unique but structured encoding for each position.

✒️ Original Embedding & Positional Encoding

Initially the **"Original Word Embedding"** shows groups for similar elements. Even after applying Positional Encoding, there is a shift in the position of the groups but the cluster remains the same.



✒️ Understanding the Relationship Between Input Embedding and Positional Encoding in k-Dimensional Space

Join @hmaracollege for more free courses🙃

In Transformers, the final input to the self-attention mechanism is the sum of two components: Final Embedding = Input Embedding + Positional Encoding

- **Input Embedding** - A k-dimensional dense vector for each token, capturing semantic meaning.
- **Positional Encoding (PE)** - A k-dimensional vector that injects positional information into the model.

📌 What are Query and Key Vectors?

Every token in the input sequence is transformed into three vectors:

- Query (Q) $\rightarrow$ Represents what this token is searching for in other tokens.
- Key (K) $\rightarrow$ Represents the properties of this token that determine whether it should be attended to.

These vectors are learned representations obtained from the original input embeddings.

$$Q = XW_Q, \quad K = XW_K$$

where:

- X is the **input embedding** matrix of size $(n \times d)$.
- W_Q, W_K are **learnable weight matrices** that transform embeddings into queries and keys.
- Q, K are the transformed matrices of size $(n \times d_k)$.

- d refers to the total embedding dimension of the model.
- d(k) represents the dimension of each attention head's query and key vectors. It is a hyperparameter.

📌 How Are They Used in Self-Attention?

The core idea of self-attention is to compare queries (Q) and keys (K) across all tokens in the sequence. The attention score for a token pair (i, j) is calculated as:

$$\text{Score}(i, j) = Q_i \cdot K_j^T$$

This dot product:

- Measures the similarity between the query of token i and the key of token j.
- If Score(i, j) is high, token j is important for token i, meaning token i should attend more to token j.

📌 Example

Note: In the Query Vectors, Key Vectors given below it's a 3*4 matrix where each row represents each word.

For example in Query Vector:

AI = [0.2 0.5 0.8 0.1]
is = [0.7 0.1 0.3 0.5]
powerful = [0.4 0.6 0.9 0.2]

Consider the sentence:

> "AI is powerful"

Step 1: Convert Each Token to Query & Key Vectors

Let's assume our embedding dimension is **4**, and we have the following transformed vectors:

Query Vectors (Q)

$$Q = \begin{bmatrix} 0.2 & 0.5 & 0.8 & 0.1 \\ 0.7 & 0.1 & 0.3 & 0.5 \\ 0.4 & 0.6 & 0.9 & 0.2 \end{bmatrix}$$

Key Vectors (K)

$$K = \begin{bmatrix} 0.3 & 0.7 & 0.2 & 0.6 \\ 0.5 & 0.2 & 0.9 & 0.4 \\ 0.8 & 0.3 & 0.6 & 0.7 \end{bmatrix}$$

Step 2: Compute Attention Scores ($Q \cdot K^T$)

To find how much attention each word pays to others, we compute the **dot product** between each query (Q) and all keys (K):

$$A = QK^T = \begin{bmatrix} (0.2, 0.5, 0.8, 0.1) \cdot K^T \\ (0.7, 0.1, 0.3, 0.5) \cdot K^T \\ (0.4, 0.6, 0.9, 0.2) \cdot K^T \end{bmatrix}$$

✒️ Why Do We Take Softmax of $Q{\cdot}KT$ in Self-Attention?

In the self-attention mechanism of Transformers, we compute the attention scores by taking the dot product of Query (Q) and Key (K) matrices:

$$A = Q \cdot K^T$$

However, this raw score matrix A needs to be transformed into a probability distribution. This is where Softmax comes in.

$$\text{Attention Weights} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

The **Softmax function** is defined as:

$$\text{softmax}(A_i) = \frac{e^{A_i}}{\sum_j e^{A_j}}$$

- **Convert Raw Scores into Probabilities** - The dot product produces unbounded values (positive or negative). Applying Softmax normalizes these values into a range [0,1] and ensures they sum to 1. This makes it possible to interpret them as relative importance scores.

- **Focus on Relevant Tokens** - A higher softmax value means more focus on that token. A lower softmax value means less focus. This enables dynamic attention, meaning the model learns which words matter more based on context.

📌 Example

Let's say we have a query-key similarity matrix:

$$A = \begin{bmatrix} 3 & 1 & 0 \\ 2 & 4 & 1 \\ 1 & 3 & 5 \end{bmatrix}$$

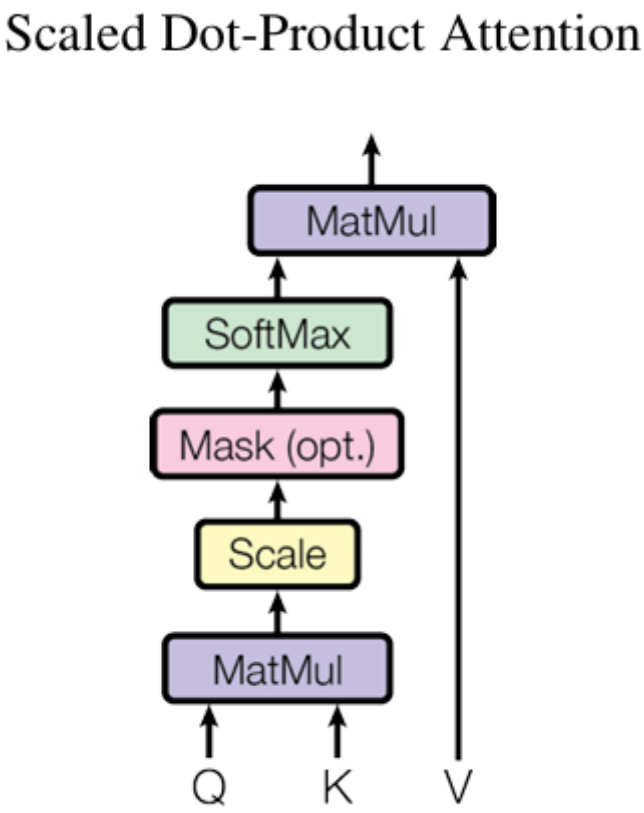
Without **softmax**, we have raw scores. After **applying softmax row-wise**, it transforms into:

$$\text{Softmax}(A) = \begin{bmatrix} 0.84 & 0.11 & 0.05 \\ 0.15 & 0.78 & 0.07 \\ 0.02 & 0.12 & 0.86 \end{bmatrix}$$

Interpretation:

- **Row 1:** Token 1 pays **84% of its attention** to itself, **11% to Token 2**, and **5% to Token 3**.
- **Row 2:** Token 2 pays **78% of its attention** to Token 2 and **only 7% to Token 3**.
- **Row 3:** Token 3 pays **86% of its attention** to Token 3.

📌 Scaled Dot-Product Attention



📌 What is the Value Vector (V) in Self-Attention?

In the self-attention mechanism of Transformers, every token in a sequence is mapped to three vectors:

- Query (Q) – Represents what a token is looking for in others.
- Key (K) – Represents what each token has to offer.
- Value (V) – Contains the actual information that will be used to generate the final output.

Unlike Query and Key, which are used to compute attention scores, Value (V) determines what content gets passed on to the next layer.

📌 How is the Value Vector Computed?

Similar to Query and Key, the Value vector is obtained by transforming the input embeddings:

$$V = XW_V$$

- X = Input embeddings of tokens ($n \times d$ matrix).
- W(V) = Learnable weight matrix for values ($d \times d(v)$).
- V = Value matrix of shape ($n \times d(v)$).

📌 Complete Example

Self-Attention Calculation for "AI is Powerful"

Step 1: Define Input Tokens & Initial Embeddings

We consider the sentence:

> **"AI is powerful"**

Each word is represented using a **word embedding vector** (arbitrarily chosen for this example):

$$X = \begin{bmatrix} 0.8 & 0.1 & 0.5 & 0.7 \\ 0.2 & 0.9 & 0.3 & 0.5 \\ 0.9 & 0.3 & 0.8 & 0.6 \end{bmatrix}$$

where each row corresponds to:

- **AI** → $[0.8, 0.1, 0.5, 0.7]$
- **is** → $[0.2, 0.9, 0.3, 0.5]$
- **Powerful** → $[0.9, 0.3, 0.8, 0.6]$

Step 2: Define Weight Matrices

Each token embedding is mapped to Query, Key, and Value vectors using learned **weight matrices**:

$$W_q = \begin{bmatrix} 0.5 & 0.2 & 0.8 & 0.3 \\ 0.1 & 0.7 & 0.2 & 0.9 \\ 0.6 & 0.3 & 0.5 & 0.7 \\ 0.4 & 0.9 & 0.1 & 0.5 \end{bmatrix}$$

$$W_k = \begin{bmatrix} 0.3 & 0.7 & 0.2 & 0.6 \\ 0.5 & 0.1 & 0.8 & 0.4 \\ 0.9 & 0.3 & 0.5 & 0.2 \\ 0.2 & 0.8 & 0.6 & 0.7 \end{bmatrix}$$

$$W_v = \begin{bmatrix} 0.7 & 0.5 & 0.8 & 0.3 \\ 0.3 & 0.9 & 0.2 & 0.7 \\ 0.6 & 0.2 & 0.9 & 0.5 \\ 0.5 & 0.8 & 0.3 & 0.6 \end{bmatrix}$$

Step 3: Compute Query, Key, and Value Matrices

We compute:

$$Q = X \cdot W_q, \quad K = X \cdot W_k, \quad V = X \cdot W_v$$

$$Q = \begin{bmatrix} 0.2 & 0.5 & 0.8 & 0.3 \\ 0.7 & 0.1 & 0.3 & 0.9 \\ 0.4 & 0.6 & 0.6 & 0.9 \end{bmatrix}$$

$$K = \begin{bmatrix} 0.3 & 0.7 & 0.2 & 0.6 \\ 0.5 & 0.2 & 0.9 & 0.4 \\ 0.8 & 0.3 & 0.6 & 0.9 \end{bmatrix}$$

$$V = \begin{bmatrix} 0.1 & 0.4 & 0.7 \\ 0.3 & 0.8 & 0.2 \\ 0.5 & 0.9 & 0.6 \end{bmatrix}$$

Step 4: Compute Attention Scores

We compute attention scores using:

$$\frac{Q \cdot K^T}{\sqrt{d_k}}$$

Since $d_k = 4$, we normalize by $\sqrt{4} = 2$.

Attention Scores (before Softmax):

$$\begin{bmatrix} 0.68 & 0.89 & 1.34 \\ 0.56 & 0.78 & 1.21 \\ 0.95 & 1.22 & 1.85 \end{bmatrix}$$

Step 5: Apply Softmax to Get Attention Weights

Softmax normalizes these scores into probabilities:

Attention Weights:

$$\begin{bmatrix} 0.23 & 0.28 & 0.49 \\ 0.20 & 0.26 & 0.54 \\ 0.15 & 0.22 & 0.63 \end{bmatrix}$$

Step 6: Compute Final Attention Output

$$\text{Output} = \text{Attention Weights} \cdot V$$

Final Attention Output:

$$\begin{bmatrix} 0.38 & 0.76 & 0.57 \\ 0.41 & 0.72 & 0.59 \\ 0.44 & 0.78 & 0.61 \end{bmatrix}$$

Conclusion

- The **weight matrices** transform embeddings into **Q, K, V**.
- **Self-attention scores** are computed using **Q and K**.
- **Softmax normalization** gives attention weights.
- The final output is a **weighted sum of Value vectors**.

This explains how **self-attention works in Transformers** for the sentence **"AI is powerful."** 🚀

✒️ Key Takeaways

- Query (Q) asks: "What am I looking for?"
- Key (K) answers: "Do I match what you're looking for?"
- Value (V) provides: "Here's the actual information you need!"

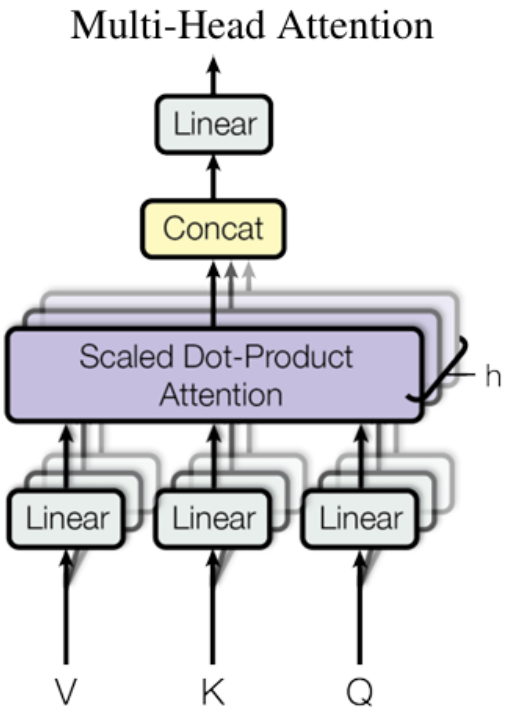
📌 Multi-Head Attention

Multi-Head Attention (MHA) is an extension of the self-attention mechanism used in Transformers. Instead of using a single attention function, the model splits the input into multiple "heads", allowing it to learn different aspects of relationships in the data simultaneously.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

✒️ Why Do We Need Multi-Head Attention?

A single self-attention mechanism can only capture one type of relationship between words at a time. However, language is complex, and different words relate to each other in multiple ways (e.g., syntactic, semantic).



✒️ Why is Transformer Architecture Known as an Encoder-Decoder Architecture?

The Transformer architecture (introduced in "Attention Is All You Need", 2017) is often referred to as an Encoder-Decoder Architecture because it consists of two main components:

- **Encoder** → Processes the input sequence and generates a meaningful representation (context).
- **Decoder** → Takes this representation and generates an output sequence (e.g., translated text).

This structure is similar to traditional sequence-to-sequence (seq2seq) models, which are commonly used for tasks like machine translation, text summarization, and question-answering.

Join @hmaracollege for more free courses🙃

💡 **Example Use Cases:**

Task	Encoder Input	Decoder Output
Translation	"I love AI"	"J'aime l'IA"
Summarization	"Transformers use self-attention..."	"Transformers rely on attention"
Question Answering	"What is AI?" + Context	"AI is..."

✒️ Steps

1. Input tokenization & embedding + Positional encoding to retain word order.
2. Multi-headed attention
    - Each token attends to every other token using a self-attention mechanism.
    - Compute Q, K and V matrices (learnt/training).
    - Different relationships - Multiple attention heads.
3. Normalization layer
4. Feedforward network - 2 fully connected layers (2 dense layers)
5. Normalization layer

Steps 2-5 are repeated. This is the overview of an Encoder.